

CHAPITRE 2

ÉTUDE DE L'EXISTANT ET DÉMARCHE DE RÉOLUTION

Introduction

Ce chapitre pose le diagnostic de la situation actuelle et présente la démarche retenue pour y répondre. Dans un premier temps, nous décrivons le fonctionnement du système en place, puis nous en ferons un diagnostic objectif en mettant en évidence ses atouts et ses insuffisances. Dans un second temps, nous présenterons la démarche adoptée pour concevoir la solution : le choix de la méthode de gestion de projet, du langage de modélisation, de l'outil retenu et des technologies de développement, chacun argumenté à travers une étude comparative rigoureuse.

1. Présentation de l'existant

1.1. Description du système actuel

La gestion des financements extérieurs à l'UJKZ repose entièrement sur des feuilles de calcul Excel. Les informations financières (montants des conventions, versements reçus, rubriques budgétaires, dépenses engagées) sont consignées dans des fichiers tenus séparément par chaque acteur, sans structure commune ni base de données partagée. Concrètement voici comment ça se passe.

Quand un bailleur signe une convention avec l'université, la DAF ouvre son tableur et y note les informations essentielles : le montant total, les rubriques budgétaires, les dates de versement et leurs modalités. Ensuite, à chaque versement reçu, un agent l'ajoute à la main au fur et à mesure.

Quand un porteur de projet veut faire une dépense, il remplit un formulaire sur papier ou par e-mail où il indique la nature de la dépense, le montant, la rubrique concernée et les justificatifs. Il envoie le dossier à la DAF. La DAF vérifie alors, dans son tableur, si l'argent est bien disponible. Puis elle transmet le dossier à l'Agent Comptable pour un second contrôle. L'Agent Comptable, de son côté, vérifie que

tout est en règle, puis enregistre le paiement dans ses propres fichiers une fois l'opération faite.

À aucun moment il n'existe un fichier commun ou un outil partagé. Chaque acteur travaille de son côté, avec ses propres chiffres, ses propres mises à jour, et sans voir ce que les autres font.

2. Les acteurs et leur rôle dans le processus

Quatre acteurs principaux interviennent dans ce processus, chacun à une étape bien précise.

Les porteurs de projet sont ceux qui lancent les demandes de dépense et fournissent les justificatifs. Mais une fois le dossier envoyé à la DAF, ils n'ont plus aucune visibilité : ils ne savent pas où en est leur demande, et doivent relancer les agents un par un pour avoir des nouvelles.

La DAF tient à jour les conventions et les versements des bailleurs. Elle gère les rubriques budgétaires et vérifie qu'une dépense est bien prévue au budget avant de la valider. C'est aussi elle qui prépare les rapports financiers pour les bailleurs. Mais pour ça, elle doit recopier et recouper manuellement les chiffres venant de ses propres fichiers et de ceux des autres acteurs. C'est long et fastidieux.

L'Agent Comptable est le deuxième niveau de contrôle. Il vérifie que le dossier est régulier d'un point de vue comptable, puis enregistre le paiement dans ses registres personnels. Lui aussi tient ses propres fichiers, sans lien avec ceux de la DAF.

Les bailleurs de fonds comme la Banque Mondiale, l'AFD, la BAD, l'UEMOA, la DDC ou l'Union Européenne financent les projets via des conventions. Ils n'interviennent pas dans la gestion quotidienne interne, mais ils imposent des règles strictes de traçabilité et de reporting. Parfois, ils règlent directement certains

prestataires sans passer par les comptes de l'université. Ces opérations doivent aussi être tracées, ce qui ajoute encore de la complexité.

3. Diagnostic de l'existant

3.1. Les points positifs

Le système actuel, aussi limité soit-il sur le plan technique, présente quelques atouts qu'il ne faudrait pas négliger.

D'abord les acteurs maîtrisent bien les procédures administratives et les règles de gestion en vigueur. Ce savoir-faire constitue une base précieuse : l'objectif est d'informatiser un processus qui fonctionne, pas d'en inventer un nouveau.

Ensuite, les conventions signées avec les bailleurs définissent un cadre clair : elles indiquent les montants, les rubriques, les versements et les obligations de reporting. Cela facilitera grandement la modélisation du futur système.

3.2. Les insuffisances

Des données dispersées et contradictoires. Chaque acteur tenant ses propres fichiers sans aucun mécanisme de synchronisation, les mêmes informations peuvent exister alors en plusieurs versions. Le solde d'une rubrique budgétaire consulté par la DAF ne correspond pas nécessairement à celui enregistré par l'Agent Comptable. Cette incohérence devient particulièrement problématique lors des audits externes, car personne ne peut certifier quelle version des données est exacte ni expliquer les écarts constatés.

Une traçabilité inexistante. Les validations ne sont pas systématiquement datées, l'identité des intervenants n'est pas toujours consignée. Rien n'empêche de modifier un fichier Excel sans laisser de trace. Cette opacité est incompatible avec les exigences des bailleurs de fonds, qui réclament une documentation complète et vérifiable de chaque opération.

Des dépassements budgétaires qui passent inaperçus. Vérifier manuellement le solde d'une rubrique dans un tableur qui n'est pas nécessairement à jour est une

opération risquée. Un oubli de mise à jour, une confusion entre deux rubriques, ou une simple erreur de saisie peuvent conduire à valider une dépense sur une rubrique déjà épuisée. Ce type d'anomalie n'est souvent découvert que lors d'un audit, bien après que la dépense ai déjà été engagée.

Un circuit de validation lent et opaque. En absence de notification automatique, un dossier peut rester plusieurs jours sans être traité. Le porteur se retrouve dans l'incertitude totale, sans aucun moyen de connaître l'état d'avancement de sa demande sans relancer manuellement chaque intervenant.

Des rapports financiers longs et peu fiables. Produire un rapport d'exécution pour un bailleur exige de collecter les données auprès de chaque acteur, de les croiser et de les consolider manuellement. La fiabilité du résultat final dépend entièrement de la rigueur avec laquelle chacun a tenu ses fichiers, une dépendance que la solution doit éliminer.

4. Démarche de résolution

4.1. Choix de la méthode de gestion de projet

La gestion d'un projet informatique ne saurait se réduire à une simple succession d'activités techniques. Elle nécessite une méthode structurée permettant d'organiser le travail, de coordonner les intervenants et de maîtriser les délais. Le choix de cette méthode est d'autant plus important que le projet CIFEU présente des caractéristiques spécifiques : des besoins qui ne sont pas tous connus dès le départ, des acteurs aux attentes parfois divergentes et une durée de réalisation limitée à trois mois.

Il existe plusieurs familles de méthodes de gestion de projet. Les méthodes classiques, comme le Cycle en V, imposent de figer l'intégralité des besoins avant de commencer le développement. Elles conviennent aux projets dont les exigences sont parfaitement stables dès le départ. Les méthodes agiles, à l'inverse, acceptent que les besoins évoluent en cours de route et privilégient des livraisons régulières de fonctionnalités opérationnelles.

Pour ce projet, notre choix s'est porté sur **Scrum** **[Annexe1]**. Trois raisons principales motivent ce choix. D'abord, les besoins n'étaient pas tous formalisés au départ : ils ont émergé progressivement au fil des échanges avec la DAF, l'Agent Comptable et les porteurs de projet. La structure en sprints courts de Scrum permet d'intégrer ces nouveaux besoins sans remettre en cause le travail déjà accompli.

Ensuite, la méthode impose des livraisons intermédiaires à l'issue de chaque sprint, ce qui nous a permis de produire régulièrement des fonctionnalités opérationnelles et de recueillir les retours des utilisateurs très tôt. Enfin, Scrum est particulièrement adapté aux projets conduits par une petite équipe, ce qui correspond précisément à notre situation.

Dans le cadre de ce projet, les rôles Scrum sont répartis comme suit :

- le **Product Owner** est M. Jean Arnauld YANOGO, Directeur du Service Informatique de l'UJKZ, responsable de la vision du produit et de la priorisation des fonctionnalités ;
- le **Scrum Master** est M. W. Jérémie OUEDRAOGO, maître de stage, garant du bon déroulement de la méthode et de la levée des obstacles ;
- l'**équipe de développement** est composée de Bonaventure Cheick Hamed OUEDRAOGO, stagiaire, qui assure seul l'ensemble de la réalisation technique.

4.2. Choix du langage de modélisation

Avant d'écrire du code, il est nécessaire de modéliser le système. La modélisation consiste à représenter la structure, les interactions et les comportements du futur logiciel sous une forme compréhensible à la fois par le développeur et par les parties prenantes non techniques. Cette étape permet de détecter les incohérences et les oublis bien avant que le développement ne commence, là où les corrections sont encore peu coûteuses.

Un langage de modélisation est un ensemble de notations graphiques standardisées permettant de représenter un système informatique de façon formelle et sans ambiguïté. Il existe plusieurs langages de modélisation graphique, mais parmi eux trois ont retenu notre attention : UML, Merise et BPMN.

Merise, méthode française née dans les années 1970, est très efficace pour concevoir des bases de données relationnelles. Cependant, elle a été conçue avant l'avènement de la programmation orientée objet et ne permet pas de représenter les comportements dynamiques du système, comme les interactions entre composants ou l'évolution des états d'un objet. BPMN, quant à lui, excelle dans la représentation des processus métier impliquant plusieurs acteurs, mais il ne couvre pas la structure interne d'un système logiciel.

Pour ce projet, notre choix s'est porté sur UML (Unified Modeling Language) [Annexe 2] et plus spécifiquement sa version 2.5.1. Trois raisons principales motivent ce choix. D'abord, UML couvre l'intégralité du cycle de conception d'un système orienté objet à travers quatorze types de diagrammes, répartis en deux familles. Les diagrammes structurels représentent la composition statique du système, à l'image du diagramme de classes qui montre les entités et leurs relations. Les diagrammes comportementaux représentent la dynamique du système, comme le diagramme de cas d'utilisation pour les fonctionnalités par acteur, le diagramme de séquence pour le déroulement chronologique des scénarios, ou encore le diagramme d'états-transitions pour l'évolution d'un objet selon les événements. Ensuite, UML est un standard internationalement reconnu, ce qui facilite la communication entre les différents acteurs du projet, du maître d'ouvrage au développeur. Enfin, il est indépendant de tout langage de programmation, ce qui nous laisse une liberté totale pour le choix des technologies d'implémentation.

Dans ce projet, nous avons produit trois types de diagrammes UML :

- les diagrammes de cas d'utilisation, qui modélisent les interactions entre les acteurs et les fonctionnalités du système ;
- les diagrammes de classes, qui représentent la structure statique avec les entités, attributs et relations ;
- les diagrammes de séquence, qui illustrent le déroulement chronologique des scénarios les plus critiques.

L'ensemble de ces diagrammes a été réalisé à l'aide de l'outil Draw.io, un logiciel de modélisation graphique en ligne, gratuit et compatible avec l'environnement Linux utilisé pour le développement.

4.3. Choix des technologies de développement

Le choix de la méthodologie et du formailisme étant fait, il nous reste à sélectionner la technologie que nous allons utiliser. Chaque composante technique a été évaluée en fonction de critères adaptés au contexte du projet : la facilité de prise en main, la

richesse fonctionnelle, les performances, la sécurité, l'existence d'une communauté active, mais aussi et surtout la compatibilité avec l'environnement technique existant à la DSI de l'UJKZ.

Backend

Le backend est la partie du système qui gère la logique métier, le stockage des données et la communication avec l'interface utilisateur. Pour ce projet, notre choix s'est porté sur Laravel 13, un framework PHP. Parmi les alternatives examinées, notamment Django avec Python et Spring Boot avec Java, Laravel s'impose pour plusieurs raisons.

Laravel intègre nativement un ORM appelé Eloquent, qui permet de manipuler les données sous forme d'objets sans écrire de requêtes SQL complexes. Son système de migrations facilite l'évolution du schéma de base de données au fil des sprints. Il propose des mécanismes de sécurité robustes contre les injections SQL, les attaques CSRF et XSS, ainsi qu'un système d'authentification prêt à l'emploi. Enfin, et c'est un argument déterminant, Laravel est le framework de référence à la DSI de l'UJKZ, ce qui garantit une intégration cohérente et facilite la maintenance future par ses équipes.

Frontend

Le frontend est la partie visible du système, avec laquelle les utilisateurs interagissent dans leur navigateur. Notre choix s'est porté sur React, associé à TypeScript. Parmi les alternatives Vue.js et Angular, plusieurs critères ont guidé notre décision.

Angular est un framework complet et très structuré, mais sa courbe d'apprentissage est plus raide et sa complexité le rend davantage adapté aux grandes équipes travaillant sur des applications d'envergure. Vue.js, quant à lui, est plus accessible et

plus léger, mais son écosystème, bien que riche, reste moins étendu que celui de React. React se distingue par plusieurs atouts. D'abord, sa flexibilité : contrairement à Angular qui impose une structure rigide, React laisse au développeur la liberté d'organiser son projet comme il l'entend, ce qui est appréciable dans un contexte de stage où les besoins peuvent évoluer. Ensuite, React bénéficie de l'écosystème le plus riche et le plus mature, avec une multitude de bibliothèques complémentaires et une communauté très active, ce qui facilite grandement la recherche de solutions aux problèmes rencontrés. Enfin, l'approche de React repose sur un flux de données unidirectionnel, qui rend le comportement de l'application plus prévisible et plus facile à déboguer. L'ajout de TypeScript apporte un typage statique qui réduit les erreurs de développement et améliore la maintenabilité du code.

Mise en forme visuelle.

Pour la mise en forme des interfaces, nous avons choisi Tailwind CSS. Cette bibliothèque propose des classes utilitaires que l'on intègre directement dans le code React, évitant ainsi la multiplication des fichiers CSS séparés. Cette approche accélère le développement et facilite la maintenance, car le style d'un composant est visible au même endroit que sa structure.

Protocole frontend-backend

La communication entre Laravel et React est assurée par **Inertia.js**, un protocole qui permet à Laravel de retourner des composants React directement depuis ses contrôleurs, sans qu'il soit nécessaire de mettre en place une API REST distincte. Cela simplifie considérablement l'architecture et réduit le volume de code à maintenir.

Base de données

Le système de gestion de base de données est le logiciel responsable du stockage, de l'organisation et de la récupération des données. Le choix est particulièrement critique dans notre contexte, car la fiabilité des données financières est une exigence absolue. Notre choix s'est porté sur PostgreSQL.

Plusieurs raisons motivent ce choix. D'abord, PostgreSQL est l'un des SGBD relationnels open source les plus robustes, avec une conformité ACID stricte. La conformité ACID, acronyme d'Atomicité, Cohérence, Isolation et Durabilité, garantit que chaque opération est soit entièrement réalisée, soit entièrement annulée en cas d'erreur. Dans un contexte financier, une transaction bancaire enregistrée à moitié ou une rubrique budgétaire mise à jour de façon incohérente est tout simplement inacceptable.

MySQL, bien que largement utilisé et offrant des garanties similaires, reste moins complet sur la gestion des transactions complexes et sur le respect strict de certaines contraintes de données. MongoDB, quant à lui, a été écarté d'emblée. Son modèle orienté documents sans schéma fixe est inadapté à des données financières structurées comme les conventions, les rubriques budgétaires et les dépenses. De plus, son niveau de conformité ACID, bien que s'étant amélioré, reste moins strict que celui des bases relationnelles.

PostgreSQL est par ailleurs déjà utilisé en production à la DSI de l'UJKZ, ce qui simplifie considérablement le déploiement et la maintenance future. Sa richesse fonctionnelle, notamment dans la gestion des transactions complexes, des clés étrangères et des contraintes d'intégrité, en fait un choix naturel pour un projet aux exigences financières strictes.

Versionnement

Dans tout projet de développement logiciel, plus la taille du code augmente, plus il devient difficile de suivre les modifications, de revenir en arrière en cas d'erreur ou de travailler sur plusieurs fonctionnalités à la fois. Pour pallier ces difficultés, nous avons utilisé Git, un système de gestion de versions distribué.

Git permet de conserver un historique complet de chaque modification du code source, d'identifier précisément qui a fait quoi et à quel moment, et de revenir à une version antérieure en cas de problème. Il offre également la possibilité de travailler sur plusieurs fonctionnalités en parallèle grâce aux branches, ce qui évite de mélanger du code en cours de développement avec du code stable. Le dépôt distant est hébergé sur GitHub, une plateforme qui assure la sauvegarde sécurisée du code et facilite le partage avec d'éventuels contributeurs.

Environnement de développement

L'environnement de développement intégré retenu est Visual Studio Code, dans sa version 1.118. Cet outil a été choisi pour sa rapidité, son intégration native avec Git et la richesse de son écosystème d'extensions.

Environnement matériel

Le développement a été réalisé sur une machine HP 15s-fq2xxx équipée d'un processeur Intel Core i3-1115G4, de 8 Go de mémoire vive et d'un disque SSD de 512 Go, sous système d'exploitation Kali Linux.

Le choix de Kali Linux, une distribution GNU/Linux basée sur Debian, repose sur plusieurs considérations. D'abord, cette distribution est reconnue pour sa stabilité et sa légèreté, ce qui est appréciable sur une machine aux ressources limitées. Ensuite, et c'est le point le plus important, elle offre une compatibilité native avec l'ensemble des outils de développement retenus, à savoir PHP, PostgreSQL, Git et Node.js.

Aucune émulation ou machine virtuelle n'est nécessaire pour exécuter ces outils, ce qui garantit des performances optimales.

Enfin, et c'est un avantage souvent sous-estimé, le serveur de production de la DSI de l'UJKZ tourne également sous Linux. Travailler sous Linux en développement garantit donc un comportement identique entre l'environnement de développement et l'environnement de production. Cette homogénéité élimine les mauvaises surprises souvent causées par des différences de configuration entre systèmes d'exploitation, notamment lors du déploiement d'applications dépendantes de bibliothèques système spécifiques.

Synthèse des choix

L'ensemble des choix méthodologiques et techniques peut être résumé comme suit. La gestion du projet est assurée par Scrum, une méthode agile adaptée aux besoins évolutifs et aux livraisons incrémentales. La modélisation du système repose sur UML, un langage standardisé qui couvre l'ensemble du cycle de conception orienté objet. Les diagrammes sont produits avec Draw.io, un outil gratuit et compatible avec l'environnement Linux.

Sur le plan technique, le backend est développé avec Laravel 13, framework de référence à la DSI de l'UJKZ. Le frontend est construit avec React et TypeScript, choisis pour leur flexibilité, leur écosystème riche et leur large adoption. La communication entre les deux est assurée par Inertia.js, qui simplifie l'architecture en évitant la mise en place d'une API REST distincte. La mise en forme visuelle est confiée à Tailwind CSS. Les données sont persistées dans PostgreSQL, SGBD robuste, conforme ACID et déjà utilisé en production à la DSI. Le versionnement est géré par Git et GitHub, et le développement s'effectue sous Visual Studio Code sur environnement Linux Kali.

Conclusion

Ce chapitre a dressé un portrait complet de la situation existante et posé les bases de la solution à construire. L'analyse du système actuel a mis en lumière ses principales limites : dispersion des données, absence de traçabilité, risques de dépassements budgétaires et lenteur du circuit de validation. La démarche de résolution a ensuite été construite pas à pas : le choix de Scrum comme méthode de gestion de projet, d'UML comme langage de modélisation, puis la sélection des technologies Laravel 13, React, Tailwind CSS et PostgreSQL. Le chapitre suivant traduit cette démarche en une analyse concrète des besoins des utilisateurs.